Beehive Monitoring Firmware
Design Summary

Date: 24 April 2026

1. Purpose of this project

This project is a proof-of-concept firmware application for a remote beehive monitor. Its purpose is to collect temperature data in low-power conditions and send summary data through the Myriota satellite platform.

2. What the firmware does (plain language)

- The device stays asleep most of the time to conserve battery power.
- Once every hour, it wakes up briefly and takes one temperature reading from the sensor.
- It stores each hourly reading in a small in-memory buffer.
- After four hourly readings are collected, it creates one message and queues it for satellite transmission.
- After this work is complete, the device returns to sleep until the next scheduled wake-up.

3. Key design choices and why they were made

- **Hourly wake-up schedule:** Matches the customer requirement and minimises active runtime.
- **Four-hour batching of data:** Reduces transmission frequency, which is typically the highest-energy operation.
- **Manual sensor read mode:** Keeps the sensor in deep sleep between readings for lower power draw.
- **Ultra-low-power sensor configuration:** Trades unnecessary precision for better long-term battery performance.
- **Integer maths instead of floating-point maths:** Reduces code size and processing cost on a small microcontroller.
- **Retry behaviour for unsent 4-hour data:** Prevents data loss if the message queue is temporarily full.
- **Failure placeholder for hourly sample:** Preserves true 4-hour windows even when a sensor read fails.
- **I2C peripheral duty-cycling:** Initialises I2C only when needed and deinitialises it afterwards to reduce leakage.

- **Calibration values cached at start-up:** Avoids repeated register reads every hour and shortens wake time.
- **Debug logging disabled by default:** Avoids runtime and binary-size overhead in production-like builds.

4. Constraints considered

- Power constraint: target operation on 4x AA batteries over a long life (up to 10 years target).
- Hardware constraint: bare-metal style environment with limited memory and compute resources.
- Communications constraint: satellite transmission is expensive in power compared with local sensing.
- Challenge scope constraint: this is a proof of concept, not a full production release.

5. What was intentionally not implemented (and why)

- **Telemetry packet-size optimisation:** Options such as packed raw values or delta encoding were assessed but not implemented. The transmission energy-per-byte model was not available, so the battery-life benefit could not be quantified confidently against added complexity.
- **Production-grade update framework in this app:** The document notes how delta updates can be supported via Myriota update APIs, but a full OTA workflow was out of scope.
- **Full production hardening:** Items like exhaustive test matrices, long-duration soak testing, and release process controls were out of scope.

6. Build and verification status

The firmware source compiles and links successfully in the current Windows 11 setup. Packaging to .bin was also validated using the documented Windows-specific build command in the README.

7. Dependency sources

- https://github.com/Myriota/Flex-SDK
- https://github.com/Myriota/Flex-Reference-Applications

8. Summary

The implementation meets the core requirements: hourly sampling, four-hour cumulative message transmission, and low-power-first behaviour. Design decisions were made to keep the solution practical for a tiny microcontroller while documenting future improvements clearly.